Compare Leaf Traversal

AlgoExpert · Very Hard · Binary Trees

The problem

- Do two trees have the same sequence of leaves left-to-right?
- Lazy iterators.

Intuition

- Walk both trees' leaves lazily in parallel so we can stop at the first difference.

Brute force vs optimal

- Collect both leaf lists, compare → O(n + m) space
- Lazy stacks → O(h1 + h2) space, early exit
- Or link leaves in place

Approach

- 1. Iterator per tree with a stack
- 2. Advance to the next leaf
- 3. Compare values
- 4. Both must end together

Worked example

```
tree 1 leaves: 4, 7, 8, 6

tree 2 leaves: 4, 7, 8, 6
```

Compare leaves left to right

Worked example

two stacks: pop until a leaf; push right then left

compare one leaf at a time

Generate leaves lazily with stacks

Worked example

```
4 = 4, 7 = 7, 8 = 8, 6 = 6, both done → true
```

All equal → true

Java solution

```java
public class CompareLeafTraversal {
    public static boolean compareLeafTraversal(BinaryTree a, BinaryTree b) {
        ArrayDeque<BinaryTree> sa = new ArrayDeque<>(), sb = new ArrayDeque<>();
        sa.push(a); sb.push(b);
        while (true) {
            BinaryTree la = nextLeaf(sa), lb = nextLeaf(sb);
            if (la == null || lb == null) return la == lb;
            if (la.value != lb.value) return false;
        }
    }

    private static BinaryTree nextLeaf(ArrayDeque<BinaryTree> st) {
        while (!st.isEmpty()) {
            BinaryTree n = st.pop();
            if (n.left == null && n.right == null) return n;
            if (n.right != null) st.push(n.right);
            if (n.left != null) st.push(n.left);
        }
        return null;
    }
}
```

Complexity

- Time: O(n + m)
- Space: O(h1 + h2)

Edge cases & pitfalls

- Different number of leaves → false
- Single-node trees
- Push right before left

Interview tips & follow-ups

- Mention early termination

Recap

- Walk both trees' leaves lazily in parallel so we can stop at the first difference.
- Complexity: O(n + m)